Práctica. Reglas propias en Wazuh y línea temporal

Nombre y apellidos: _____

Miércoles 11 de noviembre de 2026 · IC · 4 horas · trabajo individual

La semana pasada hicisteis triaje de alertas que Wazuh ya sabía generar: fallos de SSH, sudo, usuarios nuevos. Pero en una empresa real hay aplicaciones propias que Wazuh no conoce, y sus registros pasan por el SOC sin que salte nada. Hoy vais a enseñarle a Wazuh a entender una de esas aplicaciones: un portal de nóminas inventado. Escribiréis el decodificador que separa sus campos y las reglas que convierten sus líneas en alertas, incluidas dos de correlación que detectan un ataque por el patrón de varios eventos.

Con esas reglas pasaréis el registro de un día en el que pasó algo, y reconstruiréis la línea temporal del incidente juntando dos fuentes distintas. Al final probaréis vuestras reglas en vivo con vuestro propio agente.

1. Qué vas a aprender hoy

1. Explicar las tres fases por las que pasa un registro en Wazuh: predecodificación, decodificación y reglas.
2. Probar cualquier línea con `wazuh-logtest` sin tener que generarla de verdad.
3. Escribir un decodificador que extraiga campos de una aplicación propia.
4. Escribir reglas simples con `if_sid` y `field`, y reglas de correlación con `frequency`, `timeframe`, `if_matched_sid` y `same_source_ip`.
5. Cargar reglas en tu manager sin romperlo, y saber volver atrás.
6. Reconstruir una línea temporal a partir de dos registros distintos, separando lo que sabes de lo que supones.

2. Lo que necesitas

Tu Wazuh dockerizado, el mismo de las prácticas anteriores. Para la mayor parte de la práctica basta con que funcione el contenedor `wazuh.manager`; el agente solo hace falta en el apartado 10.

Si tu Wazuh no arranca, lee el apartado 11 antes de empezar: hay un plan B para hacer igualmente toda la práctica salvo la prueba en vivo.

Herramientas: `docker compose`, `python3`, `jq` y un editor de texto (`nano`, `gedit` o el que uses).

Descarga el ZIP del laboratorio de la web del curso (IC, Prácticas) y descomprímelo. Si tus carpetas se llaman de otra forma, ajusta las rutas:

```
mkdir -p ~/ic
cd ~/ic
unzip ~/Descargas/IC-2026-11-11-reglas-LAB.zip
cd IC-2026-11-11-reglas-LAB
ls
```

Lee el `README.md` entero. Ahí está el contexto de la empresa, y lo vas a necesitar para decidir qué es normal y qué no.

3. Comprueba tu manager (10 minutos)

```
cd ~/wazuh-docker/single-node
docker compose ps
```

`wazuh.manager` tiene que estar en marcha. Ahora abre la herramienta de pruebas:

```
docker compose exec wazuh.manager /var/ossec/bin/wazuh-logtest
```

Tiene que responder `Starting wazuh-logtest v4.14.8` (o tu versión) y `Type one log per line`. Pega esta línea y pulsa Intro:

```
Nov  9 08:01:44 srv-web01 sshd[901]: Accepted password for lucia from
10.20.0.31 port 51234 ssh2
```

Verás la regla `5715`, `sshd: authentication success.`, con nivel 3. Sal con `Ctrl+C`. Deja esta terminal abierta en `~/wazuh-docker/single-node` y abre otra en la carpeta del laboratorio; vas a usar las dos todo el rato.

4. Cómo procesa Wazuh una línea (20 minutos)

Lo que acabas de ver en la salida de `wazuh-logtest` son las tres fases por las que pasa cada línea que llega al manager:

1. **Predecodificación.** Wazuh reconoce la cabecera típica de syslog y separa la fecha (`timestamp`), el equipo (`hostname`) y el programa que escribió la línea (`program_name`). Esto lo hace solo, sin configurar nada.
2. **Decodificación.** Un decodificador (*decoder*) reconoce el formato concreto del mensaje y saca los campos que interesan: usuario, IP de origen, acción. Wazuh trae cientos de decodificadores para programas conocidos, como `sshd` o `sudo`.
3. **Reglas.** Las reglas miran los campos decodificados y deciden si eso merece una alerta y con qué nivel. Una regla puede colgar de otra (`if_sid`) y así se forma un árbol: primero una regla general para todo lo del programa, y debajo reglas más concretas.

Si falla la fase 2, la 3 no tiene campos con los que trabajar. Por eso, con una aplicación propia, lo primero siempre es el decodificador.

Dos normas que vas a cumplir hoy:

- Tus reglas usan identificadores entre **100000 y 120000**. Es el rango que Wazuh recomienda para reglas propias: así no chocan con las suyas, ni ahora ni cuando actualices.
- Tus decodificadores y reglas van en `local_decoder.xml` y `local_rules.xml`, dentro de `/var/ossec/etc`. Los ficheros de `/var/ossec/ruleset/` son los de Wazuh y se sobrescriben al actualizar, así que nunca se tocan. En vuestro despliegue con Docker, `/var/ossec/etc` está en un volumen, así que tus ficheros se conservan aunque reinicies el contenedor; se perderían si borrases los volúmenes.

5. Mira los registros y prueba sin decodificador (15 minutos)

En la terminal del laboratorio:

```
head -5 portal-nominas-SINTETICO.log
cat auth-srv-web01-SINTETICO.log
```

Cada línea del portal tiene esta forma:

```
Nov  9 09:12:02 srv-web01 portal-nominas[1187]: login_fail user=admin
src=203.0.113.77
```

Hay tres acciones: `login_ok`, `login_fail` y `export`. Las líneas de `export` llevan además `registros=N`, el número de registros exportados.

Vuelve a la terminal del manager, abre `wazuh-logtest` y pega esa línea. Fíjate en lo que dice la fase 1 y en lo que dice la fase 2. Apunta en la plantilla qué ha reconocido Wazuh y qué no, y por qué crees que no salta ninguna alerta.

6. Escribe el decodificador (30 minutos)

6.1. Lee el esqueleto

Abre `local_decoder.xml`. Tiene dos decodificadores:

```
<decoder name="portal-nominas">
  <program_name>^portal-nominas$</program_name>
</decoder>

<decoder name="portal-nominas-campos">
  <parent>portal-nominas</parent>
  <regex>^(\w+) user=(\S+) src=(\S+)</regex>
  <order>accion, dstuser, srcip</order>
</decoder>
```

El primero, el **padre**, solo dice "esto es mío si el programa se llama exactamente `portal-nominas`". El `^` marca el principio y el `$` el final. El segundo, el **hijo**, se aplica al mensaje de las líneas que el padre ha aceptado. En `regex`, cada paréntesis captura un trozo, y `order` dice en qué campo se guarda cada captura, por orden.

En las expresiones de Wazuh, `\w+` es una o más letras, cifras, `_`, `-` o `@`, y `\S+` es uno o más caracteres que no sean espacios. `dstuser` y `srcip` son nombres de campo que Wazuh ya conoce; `accion` es un campo nuevo que inventamos nosotros.

6.2. Súbelo al manager y pruébalo

Para copiar tus ficheros al manager tienes `subir-reglas.sh`. Ábrelo y léelo antes de ejecutarlo: nunca ejecutes un script que no sabes qué hace. Hace cuatro cosas:

1. La primera vez guarda una copia de los `local_decoder.xml` y `local_rules.xml` originales de tu manager en la carpeta del laboratorio (los ficheros `ORIGINAL-...`).
2. Copia tus dos ficheros dentro del contenedor.
3. Les pone como dueño el usuario `wazuh`.
4. Ejecuta `wazuh-analysisd -t`, que comprueba que el manager puede cargar todas las reglas y decodificadores, sin arrancar nada.

Fíjate en el orden: primero copia y después comprueba. Si la comprobación falla, tus ficheros con error ya están dentro del manager. Mientras no lo reinicies, sigue funcionando con las reglas que tenía cargadas, pero **no lo reinicies** hasta que `subir-reglas.sh` vuelva a terminar con `OK`.

Ejecútalo desde la carpeta del laboratorio:

```
./subir-reglas.sh
```

Antes de cada cambio grande, guarda también una copia de tus propios ficheros (por ejemplo, `cp local_rules.xml mis-reglas-v1.xml`). Así, si algo se tuerce, puedes volver a una versión que funcionaba.

Si termina con `OK`, vuelve a `wazuh-logtest` y pega otra vez la línea del apartado 5. `wazuh-logtest` carga las reglas cada vez que lo abres, así que no hace falta reiniciar nada para probar. Ahora la fase 2 tiene que mostrar `name: 'portal-nominas'` y los campos `accion`, `dstuser` y `srcip`, y la fase 3, la regla `100101` del esqueleto de reglas, que ya se ha subido también.

6.3. Añade el campo registros

Prueba ahora una línea de exportación:

```
Nov  9 09:17:12 srv-web01 portal-nominas[1187]: export user=lucia
src=203.0.113.77 registros=4812
```

Saca `accion`, `dstuser` y `srcip`, pero no el número de registros. Añade en `local_decoder.xml` un tercer decodificador con el **mismo nombre** que el hijo (`portal-nominas-campos`), el mismo padre y una `regex` que capture solo el número que va detrás de `registros=` en un campo llamado `registros`. Cuando dos decodificadores hijos tienen el mismo nombre, Wazuh aplica los dos y junta los campos. Para capturar cifras, usa `\d+`.

Sube y prueba hasta que la fase 2 muestre `registros: '4812'`. Comprueba también que una línea de `login_ok` sigue decodificándose bien.

7. Escribe las reglas simples (30 minutos)

Abre `local_rules.xml`. Ya tiene dos reglas:

```
<rule id="100100" level="0">
  <decoded_as>portal-nominas</decoded_as>
  <description>Portal de nóminas: evento.</description>
</rule>

<rule id="100101" level="5">
  <if_sid>100100</if_sid>
  <field name="accion">^login_fail$</field>
  <description>Portal de nóminas: acceso fallido de $(dstuser)
    desde $(srcip).</description>
  <group>authentication_failed,</group>
</rule>
```

La `100100` recoge todo lo que haya decodificado nuestro decodificador. Tiene nivel 0, así que no genera alerta: solo sirve de raíz del árbol. La `100101` cuelga de ella (`if_sid`) y solo se cumple si el campo `accion` es exactamente `login_fail`. En la descripción, `$(dstuser)` y `$(srcip)` se sustituyen por el valor de esos campos, así la alerta se entiende sin abrirla. En tu fichero la descripción va en una sola línea; aquí está partida para que quepa en la hoja.

Escribe tú, siguiendo el mismo modelo:

- **100102**, nivel 3: acceso correcto (`login_ok`). Ponle el grupo `authentication_success,`.
- **100103**, nivel 3: exportación (`export`). La descripción tiene que decir quién exporta y cuántos registros.

Sube y prueba cada regla con una línea del registro del portal. Apunta cada prueba en la tabla del apartado 3 de la plantilla: qué línea, qué regla esperabas y cuál ha salido.

Si `subir-reglas.sh` da un error, lee el mensaje: dice el fichero y la línea. Los errores más frecuentes son una etiqueta mal cerrada, una errata en el nombre de una etiqueta o un identificador repetido.

8. Escribe las reglas de correlación (45 minutos)

Una regla simple mira una línea. Una regla de correlación mira lo que ha pasado antes. Mira cómo lo hace Wazuh con SSH:

```
docker compose exec wazuh.manager grep -A8 'id="5712"' \
    /var/ossec/ruleset/rules/0095-sshd_rules.xml
```

Ejecútalo en la terminal del manager. Verás algo así (resumido):

```
<rule id="5712" level="10" frequency="8" timeframe="120" ignore="60">
  <if_matched_sid>5710</if_matched_sid>
  <same_source_ip />
  <description>sshd: brute force trying to get access to the system.
    Non existent user.</description>
```

Se lee así: "si la regla 5710 se ha cumplido 8 veces en 120 segundos, y siempre desde la misma IP de origen, salta esta regla con nivel 10". `ignore="60"` hace que, después de saltar, esta regla no genere otra alerta durante 60 segundos, para no inundar el panel; los eventos siguen llegando y analizándose. Lo que ves es la versión 4.14.8; en otras versiones la regla puede tener alguna línea distinta.

El responsable de seguridad de la empresa te pide estas tres reglas. Escríbelas tú:

Regla	Nivel	Qué tiene que detectar
100110	10	5 accesos fallidos al portal en 2 minutos desde la misma IP. Técnica MITRE `T1110`.
100111	12	Un acceso correcto desde una IP que ha disparado la 100110 en los últimos 10 minutos.
100112	10	Cualquier exportación desde una IP que no sea de la red interna 10.20.0.0/24.

Pistas:

- Para la 100110, copia la estructura de la 5712 cambiando lo necesario. Para la técnica MITRE, mira cómo la pone la 5712 en las líneas siguientes.
- La 100111 tiene que colgar de la regla de acceso correcto (`if_sid`) y, a la vez, mirar si antes se ha cumplido la 100110 (`if_matched_sid`) desde la misma IP. Como mira hacia atrás, necesita un `timeframe`, en segundos.
- Para la 100112, la etiqueta `<srcip>` admite una red con barra (`10.20.0.0/24`) y el atributo `negate="yes"`, que invierte la condición.

Prueba la 100110 pegando en `wazuh-logtest`, una detrás de otra en la misma sesión, cinco líneas de `login_fail` desde la misma IP. La quinta tiene que dar la 100110. Después pega una de `login_ok` desde esa IP y comprueba que sale la 100111. Si cierras `wazuh-logtest`, la cuenta empieza de cero.

Cuando las tres funcionen, pasa el registro entero del portal de una vez:

```
cd ~/wazuh-docker/single-node
docker compose exec -T wazuh.manager /var/ossec/bin/wazuh-logtest \
    < ~/ic/IC-2026-11-11-reglas-LAB/portal-nominas-SINTETICO.log \
    > ~/ic/salida-portal.txt
```

`-T`, como la semana pasada, hace falta porque la entrada viene de un fichero. La salida es larga; para leerla, en la terminal del laboratorio:

```
python3 resumen_logtest.py ~/ic/salida-portal.txt
python3 resumen_logtest.py ~/ic/salida-portal.txt --min-nivel 10
```

Con tus reglas bien hechas tienen que salir, entre otras, dos alertas `100112`, una `100111` y al menos una `100110`. Es posible que te salgan dos `100110`; no lo des por bueno sin más, porque es justo lo que vas a analizar en la pregunta 4.

9. Reconstruye la línea temporal (45 minutos)

Ahora vas a hacer lo mismo con las dos fuentes juntas, el portal y SSH, ordenadas por hora:

```
cd ~/ic/IC-2026-11-11-reglas-LAB
cat auth-srv-web01-SINTETICO.log portal-nominas-SINTETICO.log \
    | sort -b -k3,3 > ~/ic/srv-web01-todo.log
```

`sort -k3,3` ordena por el tercer campo, que es la hora, y `-b` ignora los espacios de más que hay delante del día. Funciona en este caso porque las dos fuentes son del mismo día, tienen el mismo formato de fecha y la misma zona horaria. No es una forma general de ordenar registros: con días, formatos o zonas distintos hay que convertir antes cada fecha a un formato común.

Pásalo por `wazuh-logtest` como en el apartado anterior, guardando la salida en `~/ic/salida-todo.txt`, y resúmelo con `resumen_logtest.py`.

Con ese resumen rellena la línea temporal de la plantilla (apartado 4). Reglas para hacerla bien:

1. Una fila por **suceso**, no por línea. Seis fallos seguidos desde la misma IP son una fila con el intervalo y el número.
2. Incluye también lo que no genera alerta grave si ayuda a entender lo que pasó. Una línea temporal no es una lista de alertas.
3. Pon la fuente de cada fila: SSH o portal.
4. Distingue lo que **sabes** (está en un registro) de lo que **supones**. Que `lucia` entró desde 203.0.113.77 lo sabes. Que no era Lucía, lo supones; ¿qué dato lo apoya?
5. La hora es local, porque estos registros no llevan zona horaria. Apúntalo en la plantilla.

Después escribe el resumen del incidente en cinco líneas como máximo, y explica cuál de las alertas de nivel 10 es un benigno esperado. El `README.md` tiene la información que necesitas.

10. Prueba tus reglas en vivo (25 minutos)

`wazuh-logtest` solo prueba: no guarda alertas. Para que tus reglas funcionen de verdad, el manager tiene que cargarlas al arrancar. Antes de reiniciarlo, **comprueba que `subir-reglas.sh` termina con `OK`**. Si reinicias el manager con un fichero roto, no arrancará y dejarás de recibir y analizar alertas hasta que lo arregles. `wazuh-analysisd -t` revisa las reglas y los decodificadores, no toda la configuración, así que después del reinicio comprueba siempre con `docker compose ps` que el manager está en marcha.

```
cd ~/wazuh-docker/single-node
docker compose restart wazuh.manager
```

Espera uno o dos minutos y comprueba que tu agente vuelve a estar `Active` con `agent_control -l`, como la semana pasada.

Ahora, **en el equipo del agente**, copia el script `simular-portal.sh` y ábrelo antes de ejecutarlo. Solo usa `logger` para escribir en el registro del sistema nueve líneas con el formato del portal, con unos segundos de pausa entre ellas. No se conecta a ningún sitio. Apunta la hora y ejecútalo:

```
date
sh simular-portal.sh
```

Espera un minuto y, en el equipo de Wazuh, busca las alertas:

```
cd ~/wazuh-docker/single-node
docker compose exec -T wazuh.manager \
    tail -n 2000 /var/ossec/logs/alerts/alerts.json \
    | jq -c 'select(.rule.groups | index("portal_nominas"))' > ~/ic/vivo.jsonl
jq -r '[.timestamp, .agent.name, .rule.id, .rule.level,
  .rule.description] | @tsv' ~/ic/vivo.jsonl
```

El primer `jq` se queda solo con las alertas cuyo grupo incluye `portal_nominas`, el que pusiste en `local_rules.xml`. Es más fiable que buscar el texto con `grep`, que también encontraría esa palabra en cualquier otra parte de la alerta.

Tienen que aparecer, con el nombre de tu agente, la 100110, la 100111 y la 100112. Fíjate en la hora: está en UTC. Rellena el apartado 5 de la plantilla con la hora local y la hora UTC. Busca también una de estas alertas en el Dashboard y haz una captura con la alerta abierta.

Si no aparece nada, revisa por orden: que el agente esté `Active`; que el manager haya arrancado (`docker compose ps`); que los mensajes estén en el equipo del agente (`sudo grep portal-nominas /var/log/syslog` en Ubuntu; si ese fichero no existe en tu sistema, `journalctl -t`

`portal-nominas`); y que el agente lea ese registro, igual que en el apartado 5.7 de la práctica anterior.

11. Si algo no funciona

`subir-reglas.sh: Permission denied`: el ZIP no ha conservado el permiso de ejecución. Dáselo con `chmod +x subir-reglas.sh simular-portal.sh`, o ejecútalos con `sh subir-reglas.sh`.

`no configuration file provided: not found`: el script no encuentra tu `docker compose`. Si tu Wazuh no está en `~/wazuh-docker/single-node`, ejecútalo así: `WAZUH_DIR=/ruta/de/tu/single-node ./subir-reglas.sh`.

`ERROR: (1226): Error reading XML file`: hay un error de XML en la línea que indica. Mira la etiqueta de esa línea y la anterior.

`wazuh-logtest error -1` o `Failure to initializing session`: has subido un fichero roto. Corrígelo y vuelve a ejecutar `subir-reglas.sh`.

La línea se decodifica, pero sale la regla 100100 y no la tuya: la condición de tu regla no se cumple. Compara letra a letra el valor del campo en la fase 2 con lo que pones en `field`.

La 100110 no salta: tienes que pegar las cinco líneas en la **misma** sesión de `wazuh-logtest`, y todas desde la misma IP.

El manager no arranca después de reiniciarlo: has reiniciado con un fichero roto. Primero guarda tu trabajo y después vuelve a dejar los originales:

```
cp local_rules.xml mis-reglas-rotas.xml
cp local_decoder.xml mi-decoder-roto.xml
cp ORIGINAL-local_rules.xml local_rules.xml
cp ORIGINAL-local_decoder.xml local_decoder.xml
./subir-reglas.sh
```

Si termina con `OK`, reinicia con `docker compose restart wazuh.manager` y comprueba con `docker compose ps` que arranca. Antes de copiar, mira con `ls -l ORIGINAL-*` que las dos copias existen y no están vacías. Si el manager no arranca y por eso no puedes copiar nada dentro, avísame.

Plan B, si tu Wazuh no arranca. Puedes arrancar un manager suelto, sin panel ni indexador, solo para probar reglas. Primero comprueba que tienes la imagen (la descargaste en la práctica 0) y que no queda un contenedor anterior con el mismo nombre:

```
docker image inspect wazuh/wazuh-manager:4.14.8 > /dev/null && echo "imagen OK"
docker ps -a --filter name=ic-logtest
```

Si la imagen no está, el plan B no te sirve sin Internet: avísame. Si ya existe un `ic-logtest` de otro día, arráncalo con `docker start ic-logtest` en vez de crearlo. Si no existe:

```
docker run -d --name ic-logtest wazuh/wazuh-manager:4.14.8
```

Espera un minuto y comprueba que responde con `docker exec -i ic-logtest /var/ossec/bin/wazuh-logtest` (sal con `Ctrl+C`). Después usa `CONTENEDOR=ic-logtest ./subir-reglas.sh` para subir las reglas, y en lugar de `docker compose exec wazuh.manager` escribe `docker exec -i ic-logtest`. Con esto puedes hacer los apartados 5 a 9, pero no el 10. Cuando termines, páralo con `docker stop ic-logtest`. Y apunta en la plantilla qué le pasa a tu Wazuh: arreglarlo es parte del itinerario de recuperación.

12. Elige tu itinerario

Recuperación

Si tu Wazuh no funciona o no hiciste la práctica del 4 de noviembre, empieza por la segunda parte de aquella hoja (la cola sintética) y después haz de esta los apartados 3 a 7 con el plan B. Lo mínimo es tener el decodificador completo y las reglas 100102 y 100103 funcionando en `wazuh-logtest`.

Ampliación

Elige una:

1. **Afina una regla.** La 100112 salta cada lunes con la exportación del director, que está autorizada. Escribe una regla 100113, hija de la 100112, que baje a nivel 3 las exportaciones desde 198.51.100.23 del usuario `director`, y solo de ese usuario. Comprueba que la exportación de `lucia` desde fuera sigue saltando con nivel 10. Explica qué riesgo aceptas al crear esta excepción.
2. **Exportaciones masivas.** Escribe una regla 100114 de nivel 8 que salte cuando alguien exporta 1.000 registros o más, desde dentro o desde fuera. Las etiquetas `field` admiten el atributo `type="pcre2"`, que permite expresiones como `^\d{4,}$` (cuatro cifras o más). Comprueba qué regla sale con la exportación de `lucia` desde fuera y explica por qué.

13. Preguntas

1. ¿Por qué no saltaba ninguna alerta con las líneas del portal antes de escribir el decodificador? ¿En qué fase se quedaban?
2. ¿Para qué sirve la regla 100100 si tiene nivel 0 y nunca genera alerta?
3. ¿Qué pasaría con la 100110 si quitaras `same_source_ip`? Pon un ejemplo con los registros del día 9.
4. `wazuh-logtest` procesa todo el fichero en menos de un segundo y no tiene en cuenta la hora escrita en cada línea. Con el registro del portal, ¿saltaría en tiempo real la **segunda** 100110 que te sale en logtest? Cuenta los fallos que caen en los 120 segundos anteriores.
5. ¿Qué intentó el atacante antes de ir al portal? ¿Lo habrías visto mirando solo el registro del portal?
6. ¿Qué dato te falta para afirmar que la persona que entró como `lucia` desde 203.0.113.77 no era Lucía?

14. Qué tienes que entregar

Una carpeta `NN-IC-reglas` comprimida en ZIP con:

1. Tus `local_decoder.xml` y `local_rules.xml` finales.
2. `plantilla-reglas.md` rellena: pruebas, línea temporal, resumen del incidente, prueba en vivo y respuestas.
3. `salida-todo.txt`, la salida de `wazuh-logtest` con las dos fuentes.
4. `vivo.jsonl` y la captura del Dashboard, o, si no has podido hacer el apartado 10, qué has comprobado y dónde te has quedado.

Si no terminas, entrega lo que tengas e indica en qué punto te has quedado.

15. Antes de entregar, comprueba que

1. `subir-reglas.sh` termina con `OK` con tus ficheros finales.
2. Todas tus reglas tienen identificador entre 100000 y 120000 y una descripción que se entiende sin abrir la alerta.
3. La 100110, la 100111 y la 100112 salen al pasar el registro completo del portal.
4. La línea temporal agrupa sucesos, indica la fuente y separa lo que sabes de lo que supones.
5. Tu manager sigue funcionando y tu agente está `Active`.
6. Ningún fichero ni captura lleva contraseñas, tokens ni datos de compañeros.

16. Seguridad

Los registros de la práctica son inventados y el script de simulación solo escribe mensajes en el registro de tu propio equipo. No lo ejecutes en equipos de otros ni en servidores del centro: aunque sean líneas falsas, en un SOC de verdad saltarían alertas y alguien tendría que investigarlas.

Las reglas que escribes hoy son de detección: avisan, no bloquean nada. Una regla mal hecha puede inundar el panel de alertas falsas o, peor, dejar pasar un ataque sin decir nada. Por eso se prueban antes con `wazuh-logtest`, como acabas de hacer, y se suben con un plan para volver atrás.

Referencias

- Wazuh, sintaxis de decodificadores: https://documentation.wazuh.com/current/user-manual/ruleset/ruleset-xml-syntax/decoders.html
- Wazuh, sintaxis de reglas: https://documentation.wazuh.com/current/user-manual/ruleset/ruleset-xml-syntax/rules.html
- Wazuh, reglas y decodificadores propios: https://documentation.wazuh.com/current/user-manual/ruleset/custom.html
- Wazuh, expresiones regulares: https://documentation.wazuh.com/current/user-manual/ruleset/ruleset-xml-syntax/regex.html
- wazuh-logtest: https://documentation.wazuh.com/current/user-manual/reference/tools/wazuh-logtest.html
- MITRE ATT&CK, T1110 Brute Force: https://attack.mitre.org/techniques/T1110/